

2. **View:** Handles business logic. For example, filtering data from Model for a specific page. The code for View is written in Python and is usually done in the *views.py* file.
3. **Template:** Handles the presentation layer to display the data provided by View in the browser. Files used for the Template are written in HTML format. You can customize filestructure the way you handle typical HTML documents, but Django provides an additional language called **DTL (Django Template Language)** used to bridge HTML and Python code.

A basic flow in the Django Basic Architecture

Let us explain the architecture with the HTTP request and response flow with key coding files to give you a more concrete idea. As the details will be explained in the following chapters, you can focus on more high-level concepts in this section.

In the actual implementation, you also need some other components of Django design architecture.

URL dispatcher: Handles URL requests to pass the request to View. It is written in the *urls.py* file.

Form: Handles user data input validations. This is optional as Django's built-in View inherits basic form functionalities.

Here are the basic steps when Django handles an HTTP request and returns an HTTP response:

1. When the **URL dispatcher** (*urls.py*) receives an HTTP request from a browser (through a web server), it defines which View (written in *view.py*) should be called.
2. The **View** called by the URL dispatcher processes the HTTP request, working with Model, Template, and Form to create an HTTP Response.